Claude Code 入門

aoki-taquan

2026 年 03 月 15 日

目次

1 はじめに 6
1.1 Claude Code とは 6
1.2 利用可能な環境 6
1.3 このガイドについて 6
1.4 対象読者 6
1.5 前提知識 6
2 インストール 7
2.1 動作要件 7
2.2 ネイティブインストール（推奨） 7
2.2.1 macOS / Linux 7
2.2.2 Windows（PowerShell） 7
2.2.3 Windows（コマンドプロンプト） 7
2.3 その他のインストール方法 7
2.3.1 Homebrew（macOS） 7
2.3.2 WinGet（Windows） 7
2.3.3 npm（非推奨） 8
2.4 インストールの確認 8
2.5 認証 8
2.5.1 認証方法の種類 8
2.5.2 認証コマンド 8
2.6 アップデート 9
2.7 Claude Code on the Web 9
3 基本操作 10
3.1 起動と終了 10
3.2 プロンプトの送信 10
3.2.1 複数行の入力 10
3.2.2 ファイル参照 10
3.2.3 ダイレクト Bash モード 10
3.3 ワンショットモード 11
3.4 会話の管理 11
3.4.1 会話の再開 11
3.4.2 セッションの選択 11
3.4.3 セッションの命名 11
3.4.4 サイドクエスチョン 11
3.5 コンテキストの管理 12
3.6 モデルの選択 12
3.7 推論の深さ（Effort Level） 12
4 ファイル操作とコード編集 13
4.1 Claude Code の内部ツール 13

4.2 ファイルの読み込み ... 13
4.3 コード検索 ... 13
4.4 ファイルの編集 ... 13
4.5 新規ファイルの作成 ... 14
4.6 複数ファイルにまたがる変更 ... 14
4.7 コマンドの実行 ... 14
4.7.1 バックグラウンド実行 ... 14
4.8 権限の管理 ... 14
4.8.1 権限モード ... 14
5 コマンドと設定 ... 16
5.1 スラッシュコマンド一覧 ... 16
5.1.1 基本コマンド ... 16
5.1.2 モデルと設定 ... 16
5.1.3 プロジェクトと会話 ... 16
5.1.4 高度な機能 ... 17
5.1.5 ビルトインスキル ... 17
5.2 CLAUDE.md による設定 ... 17
5.3 CLAUDE.md の配置場所とスコープ ... 17
5.4 自動メモリ ... 18
5.5 設定ファイル ... 18
5.5.1 設定の優先順位（高い順） ... 18
5.6 環境変数 ... 18
6 Git 連携 ... 20
6.1 基本的な Git 操作 ... 20
6.2 コミットメッセージの自動生成 ... 20
6.3 プルリクエストの作成 ... 20
6.4 コードレビュー ... 20
6.5 ブランチ操作 ... 20
6.6 ワークツリーによる並列作業 ... 21
6.7 Git の安全方針 ... 21
7 MCP（Model Context Protocol） ... 22
7.1 MCP とは ... 22
7.2 MCP サーバーの追加 ... 22
7.2.1 HTTP サーバー（推奨） ... 22
7.2.2 Stdio サーバー（ローカル実行） ... 22
7.2.3 SSE サーバー（非推奨） ... 22
7.3 サーバーのスコープ ... 22
7.4 サーバーの管理 ... 23
7.5 活用例 ... 23
8 IDE 連携 ... 24
8.1 VS Code 拡張機能 ... 24
8.1.1 インストール ... 24

8.1.2 主な機能 ... 24
8.1.3 キーボードショートカット ... 24
8.2 JetBrains プラグイン ... 24
8.2.1 対応 IDE ... 24
8.2.2 インストール ... 24
8.2.3 主な機能 ... 24
8.3 Claude Desktop 連携 ... 25
8.4 ターミナルからの利用 ... 25
8.5 リモートコントロール ... 25
9 高度な使い方 ... 26
9.1 Hooks（フック） ... 26
9.1.1 フックイベントの種類 ... 26
9.1.2 フックの設定 ... 26
9.1.3 フックの種類 ... 27
9.2 サブエージェント ... 27
9.2.1 ビルトインエージェント ... 27
9.2.2 カスタムサブエージェントの作成 ... 27
9.3 ヘッドレスモード ... 27
9.3.1 出力フォーマット ... 27
9.4 マルチターンパイプライン ... 27
9.5 カスタムスラッシュコマンド ... 28
9.6 大規模変更（/batch） ... 28
9.7 権限の細かい制御 ... 28
9.8 Claude Agent SDK ... 29
10 Tips とベストプラクティス ... 30
10.1 効果的なプロンプトの書き方 ... 30
10.2 キーボードショートカット ... 30
10.2.1 基本操作 ... 30
10.2.2 表示・モード ... 30
10.2.3 実行制御 ... 30
10.2.4 テキスト編集 ... 31
10.3 コスト管理 ... 31
10.4 CLAUDE.md を活用する ... 31
10.5 実践的なワークフロー ... 31
10.5.1 新しいコードベースの理解 ... 31
10.5.2 バグ修正 ... 32
10.5.3 PR の作成 ... 32
10.6 よくある問題と対処法 ... 32
10.6.1 コンテキストが大きすぎる ... 32
10.6.2 権限確認が頻繁 ... 32
10.6.3 Claude が古い情報を参照する ... 32
10.6.4 接続エラー ... 32

10.7 クイックリファレンス 32
10.7.1 必須コマンド 32
10.7.2 必須ショートカット 33
10.7.3 3 つの基本パターン 33

1 はじめに

1.1 Claude Code とは

Claude Code は、Anthropic が提供するターミナルベースの AI コーディングアシスタントです。コマンドラインから直接 Claude を呼び出し、コードの読み書き、コマンドの実行、Git 操作、ファイル検索など、ソフトウェア開発に必要な作業を自然言語で指示できます。

主な特徴は以下の通りです：

- **エージェント型**：ファイルの読み取り、判断、操作、フィードバックからの学習を自律的に行う
- **ターミナルネイティブ**：エディタやブラウザを離れずに AI と対話できる
- **コードベース理解**：プロジェクト全体を把握し、文脈に沿った回答・編集を行う
- **ツール実行**：シェルコマンドの実行、ファイルの読み書き、Git 操作を直接行う
- **安全性**：ユーザーの許可なく破壊的な操作を行わない権限管理システム
- **マルチプラットフォーム**：macOS、Linux、Windows（WSL 含む）に対応
- **IDE 連携**：VS Code、JetBrains IDE、Claude Desktop との統合

1.2 利用可能な環境

Claude Code は以下の環境で動作します：

- **ローカルターミナル**：macOS、Linux、Windows のターミナルから直接利用
- **Claude Code on the Web**：ブラウザ上で動作するクラウド環境
- **IDE 拡張機能**：VS Code や JetBrains IDE 内で統合利用
- **Claude Desktop**：デスクトップアプリとの連携

1.3 このガイドについて

本書は Claude Code を初めて使う方を対象に、インストールから基本操作、 高度な活用法までを網羅的に解説する入門書です。

1.4 対象読者

- ターミナル操作に慣れている開発者
- AI を開発ワークフローに取り入れたい方
- Claude Code をこれから使い始める方

1.5 前提知識

- 基本的なターミナル操作（`cd`、`ls` など）
- Git の基本概念（コミット、ブランチなど）
- いずれかのプログラミング言語の基礎知識

2 インストール

2.1 動作要件

Claude Code を利用するには以下が必要です：

- **OS**：macOS 13.0 以降、Ubuntu 20.04+ / Debian 10+、Windows 10/11
- **メモリ**：4GB 以上の RAM
- **ネットワーク**：インターネット接続（必須）
- **Anthropic アカウント**：Max、Team、Enterprise、または Console アカウント

2.2 ネイティブインストール（推奨）

自動アップデート機能付きのネイティブインストーラが推奨です：

2.2.1 macOS / Linux

```
curl -fsSL https://claude.ai/install.sh | bash
```

2.2.2 Windows（PowerShell）

```
irm https://claude.ai/install.ps1 | iex
```

2.2.3 Windows（コマンドプロンプト）

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del
install.cmd
```

2.3 その他のインストール方法

2.3.1 Homebrew（macOS）

```
brew install --cask claude-code
```

手動アップデートが必要です（`brew upgrade --cask claude-code`）。

2.3.2 WinGet（Windows）

```
winget install Anthropic.ClaudeCode
```

2.3.3 npm（非推奨）

```
npm install -g @anthropic-ai/claude-code
```

npm 経由は非推奨になりました。ネイティブインストーラへの移行を推奨します。

2.4 インストールの確認

```
claude --version
```

問題がある場合は診断コマンドを実行します：

```
claude doctor
```

2.5 認証

初回起動時に認証が必要です：

```
claude
```

ブラウザが開くので Anthropic アカウントでログインします。

2.5.1 認証方法の種類

方法	説明
Anthropic アカウント	デフォルト。ブラウザ認証
API キー	`ANTHROPIC_API_KEY` 環境変数を設定
Amazon Bedrock	`CLAUDE_CODE_USE_BEDROCK=1` を設定
Google Vertex AI	`CLAUDE_CODE_USE_VERTEX=1` を設定
SSO	`claude auth login --sso` で組織の SSO を利用

API キーを使う場合：

```
export ANTHROPIC_API_KEY="sk-ant-..."
claude
```

2.5.2 認証コマンド

```
claude auth login     # ログイン
claude auth logout    # ログアウト
claude auth status    # 認証状態を確認
```

2.6 アップデート

ネイティブインストールの場合は自動アップデートされます。 手動で更新する場合：

```
claude update
```

2.7 Claude Code on the Web

ブラウザ上で Claude Code を利用する方法もあります。 claude.ai にアクセスし、Claude Code を有効にすると、 Web 上のターミナル環境で同様の操作が可能です。

- ローカル環境のセットアップが不要
- すぐに試すことができる
- `/teleport` でターミナルセッションに移行可能

3 基本操作

3.1 起動と終了

プロジェクトディレクトリで `claude` を実行すると対話モードが始まります：

```
cd your-project
claude
```

初回起動時には初期プロンプトを指定することもできます：

```
claude "このプロジェクトの構成を説明して"
```

終了するには以下のいずれかを使います：

- `/exit` と入力
- `Ctrl+C` を 2 回押す
- `Ctrl+D`（EOF）を送信

3.2 プロンプトの送信

起動後、自然言語でプロンプトを入力します：

```
> このプロジェクトの構成を説明して
```

3.2.1 複数行の入力

複数行のプロンプトを書きたい場合：

方法	対応ターミナル
`Shift+Enter`	iTerm2, WezTerm, Ghostty, Kitty
`Option+Enter`	macOS デフォルト
`\` + `Enter`	すべてのターミナル
`Ctrl+J`	すべてのターミナル

3.2.2 ファイル参照

`@` でファイルやフォルダを参照できます（オートコンプリート対応）：

```
> @src/auth.ts のバグを修正して
```

3.2.3 ダイレクト Bash モード

`!` で始めるとシェルコマンドを直接実行できます：

```
> !npm test
```

3.3 ワンショットモード

対話モードに入らず、単発でコマンドを実行できます：

```
claude -p "このリポジトリのREADMEを要約して"
```

`-p`（`--print`）フラグを使うと結果を標準出力に表示して終了します。パイプとの組み合わせも可能です：

```
cat error.log | claude -p "このエラーの原因を分析して"
```

出力形式を指定できます：

```
claude -p "テストを実行して" --output-format json
```

3.4 会話の管理

3.4.1 会話の再開

前回の会話を再開するには `--continue`（`-c`）フラグを使います：

```
claude -c
```

3.4.2 セッションの選択

過去の会話を一覧から選んで再開する場合は `--resume` を使います：

```
claude --resume
```

3.4.3 セッションの命名

セッションに名前を付けると後で見つけやすくなります：

```
claude -n "auth-refactor"
```

3.4.4 サイドクエスチョン

本筋の会話を汚さずに質問したい場合は `/btw` を使います：

```
/btw TypeScript の enum と const enum の違いは？
```

3.5 コンテキストの管理

Claude Code は会話が長くなるとコンテキストウィンドウの上限に近づきます。

- **自動圧縮**：古いメッセージは自動的に要約・圧縮される
- **手動圧縮**：`/compact` コマンドで圧縮
- **クリア**：`/clear` で会話をリセット
- **コンテキスト確認**：`/context` で現在の使用量を確認
- **巻き戻し**：`Esc` を 2 回押すとコードや会話を巻き戻せる

3.6 モデルの選択

`/model` コマンドまたは起動時フラグでモデルを切り替えられます：

エイリアス	モデル	用途
`sonnet`	Sonnet 4.6	日常的なタスク、バランス重視
`opus`	Opus 4.6	複雑な推論、高品質
`haiku`	Haiku 4	軽いタスク、低コスト
`sonnet[1m]`	Sonnet + 1M コンテキスト	大規模コードベース
`opus[1m]`	Opus + 1M コンテキスト	複雑な大規模プロジェクト

```
# 起動時に指定
claude --model opus

# セッション中に切り替え
/model sonnet
```

3.7 推論の深さ（Effort Level）

Claude がどの程度深く考えるかを調整できます：

- `low`：高速、簡単なタスク向け
- `medium`：標準
- `high`：複雑なタスク向け
- `max`：最大限の推論（Opus 4.6 のみ）

```
claude --effort high
```

セッション中は `/effort` コマンドで変更できます。 拡張思考のオン/オフは `Alt+T`（macOS: `Option+T`）で切り替えられます。

4 ファイル操作とコード編集

4.1 Claude Code の内部ツール

Claude Code は内部的に複数のツールを持ち、タスクに応じて使い分けます：

ツール	機能
Read	ファイルの読み取り（テキスト、画像、PDF 対応）
Edit	既存ファイルの差分ベース編集
Write	新規ファイルの作成
Glob	ファイル名パターンでの検索
Grep	ファイル内容の正規表現検索
Bash	シェルコマンドの実行
Agent	サブエージェントの起動
WebFetch	URL からのコンテンツ取得
WebSearch	Web 検索

4.2 ファイルの読み込み

```
> src/main.ts の内容を見せて
```

Claude は `Read` ツールでファイルを読み取ります。 テキストファイルだけでなく、画像（PNG、JPG）や PDF も読み込み可能です。

`@` を使ったファイル参照も便利です：

```
> @src/components/Button.tsx のこのコンポーネントを説明して
```

4.3 コード検索

プロジェクト内のコードを検索する際、Claude は内部的に `Glob` と `Grep` を使い分けます：

```
> "handleSubmit" という関数がどこで定義されているか探して
```

```
> src/ 以下の全ての .tsx ファイルを一覧にして
```

4.4 ファイルの編集

コードの修正を依頼すると、Claude は `Edit` ツールで差分ベースの編集を行います：

```
> handleSubmit 関数にバリデーションを追加して
```

編集内容は差分（diff）として表示され、ユーザーが確認・承認できます。 編集前に必ずファイルの内容を確認してから、最小限の変更を行います。

4.5 新規ファイルの作成

```
> src/utils/validation.ts にメールアドレスのバリデーション関数を作って
```

Claude は `Write` ツールを使ってファイルを作成します。

4.6 複数ファイルにまたがる変更

リファクタリングのように複数ファイルに影響する変更も得意です：

```
> UserService クラスを UserRepository と UserUseCase に分割して
```

Claude はプロジェクト全体の構造を理解した上で、関連するファイルを一括で変更します。

4.7 コマンドの実行

シェルコマンドの実行を依頼できます：

```
> npm test を実行して
```

4.7.1 バックグラウンド実行

長時間かかるコマンドは `Ctrl+B` でバックグラウンド実行に切り替えられます。完了時に通知されます。

4.8 権限の管理

ファイルの読み書きやコマンドの実行には権限が必要です。 Claude が操作を行う前にユーザーに確認を求めます：

- **Allow once**：今回のみ許可
- **Allow for this session**：このセッション中は許可
- **Always allow**：常に許可（設定に保存）
- **Deny**：拒否

4.8.1 権限モード

`Shift+Tab` で権限モードを切り替えられます：

モード	動作
`default`	各ツール使用時に確認を求める
`plan`	読み取りのみ。編集前にプランを提示
`acceptEdits`	ファイル編集は自動承認、Bash は確認
`dontAsk`	事前承認されたもの以外は自動拒否
`bypassPermissions`	全チェックをスキップ（危険）

```
# 起動時に指定
claude --permission-mode plan
```

5 コマンドと設定

5.1 スラッシュコマンド一覧

Claude Code では `/` で始まるスラッシュコマンドが利用できます。 `/` を入力するとコマンドメニューが表示されます。

5.1.1 基本コマンド

コマンド	説明
`/help`	ヘルプを表示
`/exit`	Claude Code を終了
`/clear`	会話履歴をクリア
`/compact`	会話を要約して圧縮
`/context`	現在のコンテキスト使用量を表示
`/status`	アカウントとセッション情報を表示
`/cost`	トークン使用量とコストを表示

5.1.2 モデルと設定

コマンド	説明
`/model`	モデルを切り替え
`/effort`	推論の深さを調整（low/medium/high/max）
`/permissions`	権限設定を管理
`/config`	設定を変更
`/terminal-setup`	ターミナルのキーボードショートカットを設定
`/vim`	Vim モードの有効/無効
`/theme`	出力テーマをカスタマイズ
`/statusline`	ステータスラインの表示設定

5.1.3 プロジェクトと会話

コマンド	説明
`/init`	CLAUDE.md を初期化・再生成
`/memory`	CLAUDE.md と自動メモリを表示・編集
`/resume`	過去の会話を選択して再開
`/rename`	現在のセッション名を変更
`/add-dir`	セッション中に作業ディレクトリを追加
`/btw`	履歴に残らないサイドクエスチョン

5.1.4 高度な機能

コマンド	説明
`/agents`	サブエージェントの作成・管理
`/mcp`	MCP サーバーの管理
`/hooks`	フックの確認・管理
`/ide`	IDE に接続（VS Code, JetBrains）
`/desktop`	セッションを Claude Desktop に引き渡し
`/teleport`	Web セッションをターミナルに移行

5.1.5 ビルトインスキル

コマンド	説明
`/batch <指示>`	大規模な並列変更を実行
`/simplify`	直近の変更をレビュー・改善
`/debug`	問題のトラブルシューティング
`/loop <間隔> <プロンプト>`	プロンプトを定期実行

5.2 CLAUDE.md による設定

`CLAUDE.md` はプロジェクトのルートに置く設定ファイルです。 Claude Code が作業する際の規約や手順を自然言語で記述します：

```
# CLAUDE.md

## プロジェクト概要
TypeScript + React のフロントエンドアプリケーション。

## ビルド・テスト
- ビルド: npm run build
- テスト: npm test
- リント: npm run lint

## 規約
- 関数名はキャメルケース
- コンポーネントは関数コンポーネントで書く
- テストは __tests__/ に配置
```

`/init` コマンドで Claude に自動生成させることもできます。

5.3 CLAUDE.md の配置場所とスコープ

場所	スコープ
`./CLAUDE.md`	プロジェクト共有（Git 管理推奨）

場所	スコープ
`.claude/CLAUDE.md`	プロジェクトの個人設定
`~/.claude/CLAUDE.md`	全プロジェクト共通の個人設定
`/etc/claude-code/CLAUDE.md`	組織全体（管理者設定）

5.4 自動メモリ

Claude Code はセッション中に学んだことを自動的に記憶します。

- デフォルトで有効
- `~/.claude/projects/<project>/memory/` に保存
- セッション開始時に最初の 200 行が読み込まれる
- `/memory` コマンドで確認・編集可能

5.5 設定ファイル

`.claude/settings.json` でプロジェクトレベルの設定を管理します：

```
{
  "model": "sonnet",
  "effortLevel": "high",
  "permissions": {
    "allow": ["Read", "Bash(npm run *)"],
    "deny": ["Bash(rm *)"]
  }
}
```

5.5.1 設定の優先順位（高い順）

1. 管理ポリシー（組織設定）
2. CLI 引数（`--allowedTools` など）
3. プロジェクトローカル（`.claude/settings.local.json`）
4. プロジェクト共有（`.claude/settings.json`）
5. ユーザー設定（`~/.claude/settings.json`）

5.6 環境変数

Claude Code の動作に影響する主な環境変数：

変数	説明
`ANTHROPIC_API_KEY`	API キー
`CLAUDE_CODE_USE_BEDROCK`	Amazon Bedrock を使用
`CLAUDE_CODE_USE_VERTEX`	Google Vertex AI を使用
`ANTHROPIC_MODEL`	デフォルトモデルを指定

変数	説明
`CLAUDE_CODE_MAX_TURNS`	最大ターン数を制限

6 Git 連携

6.1 基本的な Git 操作

Claude Code は Git リポジトリ内で動作している場合、Git 操作を直接実行できます：

```
> 変更をコミットして
```

Claude は `git status` で変更を確認し、適切なコミットメッセージを生成してコミットします。

6.2 コミットメッセージの自動生成

変更内容を分析して意味のあるコミットメッセージを自動生成します：

```
> 今の変更をコミットして。日本語でメッセージを書いて
```

Claude は差分を確認し、変更の意図を反映したメッセージを作成します。

6.3 プルリクエストの作成

GitHub の PR 作成も支援します：

```
> この変更でPRを作成して
```

Claude は `gh` コマンドを使い、変更内容をまとめたタイトルと説明を生成して PR を作成します。PR はセッションにリンクされるため、後から再開できます：

```
claude --from-pr 123
```

6.4 コードレビュー

既存の PR のレビューも可能です：

```
> PR #42 をレビューして
```

`/review` コマンドで現在のブランチの変更をレビューすることもできます。

6.5 ブランチ操作

ブランチの作成や切り替えも自然言語で指示できます：

```
> feature/add-auth というブランチを作って切り替えて
```

6.6 ワークツリーによる並列作業

`--worktree` フラグで Git ワークツリーを使った隔離環境を作成できます：

```
claude --worktree feature-x
```

これにより、現在の作業ディレクトリに影響を与えず、 別のブランチで並行して作業できます。

6.7 Git の安全方針

Claude Code は Git 操作に関して以下の安全方針を持っています：

- **確認を求める操作**：`git push --force`、`git reset --hard`、`git checkout .` などの破壊的操作
- **コミット前の確認**：変更内容を表示してからコミット
- **機密ファイルの保護**：`.env` や認証情報ファイルをコミットしないよう注意
- **新規コミット優先**：`--amend` は明示的に指示された場合のみ使用
- **フックの尊重**：`--no-verify` を使わない（明示的な指示がない限り）

7 MCP（Model Context Protocol）

7.1 MCP とは

MCP（Model Context Protocol）は、AI モデルが外部のツールやデータソースと 連携するための標準プロトコルです。Claude Code は MCP クライアントとして動作し、 MCP サーバーが提供するツールを利用できます。

7.2 MCP サーバーの追加

7.2.1 HTTP サーバー（推奨）

```
claude mcp add --transport http <名前> <URL>
```

例：Notion との連携

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

7.2.2 Stdio サーバー（ローカル実行）

```
claude mcp add --transport stdio <名前> -- <コマンド> [引数]
```

例：Airtable との連携

```
claude mcp add --transport stdio airtable \
  --env AIRTABLE_API_KEY=YOUR_KEY \
  -- npx -y airtable-mcp-server
```

7.2.3 SSE サーバー（非推奨）

```
claude mcp add --transport sse <名前> <URL>
```

7.3 サーバーのスコープ

MCP サーバーには 3 つのスコープがあります：

スコープ	説明
Local	自分だけ、現在のプロジェクトのみ（`~/.claude.json`）
Project	チーム共有（`.mcp.json` をリポジトリに含める）
User	自分の全プロジェクト（`~/.claude.json`）

スコープの指定：

```
claude mcp add my-server -s local -- npx my-server
claude mcp add my-server -s project -- npx my-server
claude mcp add my-server -s global -- npx my-server
```

7.4 サーバーの管理

```
# サーバー一覧を表示
claude mcp list

# サーバーの詳細を表示
claude mcp get my-server

# サーバーを削除
claude mcp remove my-server
```

対話モード内では `/mcp` コマンドでサーバーの状態を確認・管理できます。

7.5 活用例

MCP サーバーを活用すると以下のような連携が可能になります：

- **GitHub**：コードレビュー、PR 管理
- **Jira / Linear**：イシュートラッキング
- **データベース**：PostgreSQL、BigQuery へのクエリ
- **Sentry**：エラーモニタリング
- **Slack**：メッセージの送受信
- **Playwright**：ブラウザ自動操作
- **Notion**：ドキュメント管理

8 IDE 連携

8.1 VS Code 拡張機能

8.1.1 インストール

VS Code のコマンドパレット（`Cmd+Shift+X` / `Ctrl+Shift+X`）で “Claude Code” を検索してインストールします。Cursor IDE にも対応しています。

8.1.2 主な機能

- **インライン diff ビューア**：変更内容を IDE 内で確認
- **ファイル参照**：`@` でファイルやフォルダを行番号付きで参照
- **コマンドパレット統合**：IDE のコマンドパレットから操作
- **マルチタブ**：複数の会話を同時に管理
- **プランレビュー**：実行前にプランを確認
- **権限モード切替**：UI から権限モードを変更
- **MCP サーバー管理**：`/mcp` コマンドで管理
- **会話の再開**：過去の会話を選択して再開

8.1.3 キーボードショートカット

ショートカット	動作
`Cmd+Esc` / `Ctrl+Esc`	エディタと Claude のフォーカスを切替
`Option+K` / `Alt+K`	`@` 参照を行番号付きで挿入
`Cmd+Shift+Esc` / `Ctrl+Shift+Esc`	新しいタブで開く
`Cmd+N` / `Ctrl+N`	新しい会話を開始

8.2 JetBrains プラグイン

8.2.1 対応 IDE

IntelliJ IDEA、PyCharm、WebStorm、PhpStorm、GoLand、Android Studio などの JetBrains IDE に対応しています。

8.2.2 インストール

JetBrains IDE のプラグインマーケットプレイスから “Claude Code” をインストールします。

8.2.3 主な機能

- **クイック起動**：`Cmd+Esc` / `Ctrl+Esc`
- **IDE diff ビューア統合**：変更差分を IDE 内で確認

- **選択コンテキスト共有**：選択したコードを Claude に送信
- **ファイル参照**：`Cmd+Option+K` / `Alt+Ctrl+K`
- **診断エラー共有**：IDE のエラーを Claude に直接共有

8.3 Claude Desktop 連携

`/desktop` コマンドで現在のセッションを Claude Desktop アプリに引き渡せます。逆に、Web セッションからターミナルへの移行は `/teleport` を使います。

8.4 ターミナルからの利用

IDE を使わない場合でも、任意のターミナルエミュレータから利用できます。 tmux や screen と組み合わせると、バックグラウンドで長時間のタスクを 実行させることも可能です：

```
tmux new-session -s claude
claude
```

8.5 リモートコントロール

別のデバイス（スマートフォンやブラウザ）からターミナルセッションを操作できます：

```
# リモートコントロールサーバーを起動
claude remote-control --name "My Session"

# 別のデバイスから接続
claude --remote-control
```

9 高度な使い方

9.1 Hooks（フック）

Hooks は Claude Code のライフサイクルの特定のタイミングでカスタムスクリプトを実行する仕組みです。

9.1.1 フックイベントの種類

イベント	タイミング
`SessionStart`	セッション開始・再開時
`UserPromptSubmit`	ユーザーがプロンプトを送信した時
`PreToolUse`	ツール実行前
`PostToolUse`	ツール実行成功後
`PostToolUseFailure`	ツール実行失敗後
`PermissionRequest`	権限ダイアログ表示時
`Notification`	Claude が入力待ちの時
`SubagentStart/Stop`	サブエージェントのライフサイクル
`Stop`	Claude が応答を完了した時
`PreCompact/PostCompact`	コンテキスト圧縮の前後
`ConfigChange`	設定ファイル変更時

9.1.2 フックの設定

`.claude/settings.json` にフックを定義します：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write $CLAUDE_FILE_PATH"
          }
        ]
      }
    ]
  }
}
```

この例ではファイル編集後に自動的に Prettier でフォーマットします。

9.1.3 フックの種類

- **Command**：シェルコマンドを実行
- **HTTP**：外部エンドポイントに POST
- **Prompt**：LLM で yes/no 判定
- **Agent**：ツール付きのマルチターン検証

9.2 サブエージェント

サブエージェントは隔離されたコンテキストウィンドウで動作する特殊なエージェントです。

9.2.1 ビルトインエージェント

- **Explore**：読み取り専用、高速なコードベース探索
- **Plan**：実装計画の設計
- **General-purpose**：汎用的なタスク実行

9.2.2 カスタムサブエージェントの作成

`/agents` コマンドで独自のサブエージェントを作成できます：

1. スコープ（User / Project）を選択
2. 目的を記述
3. 利用可能なツールを選択
4. モデルを指定

各サブエージェントには独自のシステムプロンプト、ツール制限、モデル設定が可能です。

9.3 ヘッドレスモード

CI/CD パイプラインやスクリプトから非対話的に実行できます：

```
claude -p "テストを実行して結果を報告して" \
  --output-format json \
  --max-turns 10
```

9.3.1 出力フォーマット

- `text`：プレーンテキスト（デフォルト）
- `json`：JSON 形式
- `stream-json`：ストリーミング JSON

9.4 マルチターンパイプライン

複数のプロンプトを連鎖させて複雑なワークフローを構築できます：

```
# 分析結果をファイルに保存
claude -p "このプロジェクトの問題点を分析して" \
  --output-format json > analysis.json

# 結果を使って次の処理
cat analysis.json | claude -p "この分析をもとに修正して" \
  --continue
```

9.5 カスタムスラッシュコマンド

プロジェクト固有のスラッシュコマンドを定義できます。`.claude/commands/` にマークダウンファイルを配置します：

```
.claude/commands/review.md
```

ファイルの内容がプロンプトテンプレートになります：

```
以下の観点でコードレビューを行ってください：
1. セキュリティ上の問題
2. パフォーマンスの問題
3. 可読性の改善点

対象: $ARGUMENTS
```

`/project:review src/auth.ts` のように呼び出せます。

9.6 大規模変更（/batch）

`/batch` コマンドで大規模な並列変更を実行できます：

```
/batch src/ 以下の jQuery を React に移行して
```

Claude は並列プランを作成し、各サブエージェントがそれぞれの変更を担当します。

9.7 権限の細かい制御

権限ルールはグロブパターンや正規表現で細かく指定できます：

```
{
  "permissions": {
    "allow": [
      "Read",
      "Bash(npm run *)",
      "Edit(/src/**/*.ts)",
      "WebFetch(domain:github.com)",
```

```
      "Agent(Explore)"
    ],
    "deny": [
      "Bash(rm *)"
    ]
  }
}
```

9.8 Claude Agent SDK

Claude Code の機能をプログラムから利用するための SDK です：

```
import { Agent } from "claude-agent-sdk";

const agent = new Agent({
  model: "claude-sonnet-4-5-20250514",
});

const result = await agent.run("テストを実行して");
```

10 Tips とベストプラクティス

10.1 効果的なプロンプトの書き方

Claude Code を効果的に使うためのコツ：

- **具体的に指示する**：「コードを改善して」より「handleSubmit に null チェックを追加して」
- **コンテキストを与える**：`@` でファイルを参照し、背景情報を伝える
- **段階的に進める**：大きなタスクは複数のステップに分割する
- **ファイルパスを明示する**：「`src/utils/auth.ts` の `validateToken` を修正して」
- **Plan モードで探索**：まず Plan モードで調査し、理解してから編集に進む

10.2 キーボードショートカット

10.2.1 基本操作

ショートカット	動作
`Enter`	メッセージを送信
`Shift+Enter`	改行
`Ctrl+C`	現在の生成を中断 / 2 回で終了
`Ctrl+D`	セッションを終了
`Ctrl+L`	画面クリア（履歴は保持）
`Esc` × 2	巻き戻し / 要約
`上矢印`	直前のプロンプトを呼び出す

10.2.2 表示・モード

ショートカット	動作
`Ctrl+O`	詳細出力の切替（思考過程を表示）
`Shift+Tab`	権限モードを切替
`Alt+P` / `Option+P`	モデルを切替
`Alt+T` / `Option+T`	拡張思考の切替
`Ctrl+T`	タスクリストの表示切替

10.2.3 実行制御

ショートカット	動作
`Ctrl+B`	バックグラウンド実行に切替
`Ctrl+F` × 2	バックグラウンドエージェントを停止
`Ctrl+G`	テキストエディタでプロンプトを編集

ショートカット	動作
`Ctrl+V`	クリップボードから画像を貼り付け

10.2.4 テキスト編集

ショートカット	動作
`Ctrl+K`	行末まで削除
`Ctrl+U`	行全体を削除
`Ctrl+Y`	削除したテキストを貼り付け
`Alt+B` / `Alt+F`	単語単位でカーソル移動
`Ctrl+R`	履歴の逆方向検索

カスタムショートカットは `~/.claude/keybindings.json` で設定できます。

10.3 コスト管理

Claude Code はトークンを消費します。コストを意識するためのヒント：

- `/cost` でセッション中の使用量を確認
- `--max-turns` で自動実行のターン数を制限
- `/compact` で会話を圧縮してトークンを節約
- 簡単なタスクには `haiku` モデルを使う
- 大きなファイルは全体ではなく必要な部分だけ読ませる
- 推論の深さ（effort）を調整する

10.4 CLAUDE.md を活用する

プロジェクト固有の情報を CLAUDE.md に記述しておくと、 毎回同じ説明をする必要がなくなります：

- ビルドコマンド、テストコマンド
- コーディング規約
- ディレクトリ構成の説明
- よく使うワークフロー
- 禁止事項や注意点

10.5 実践的なワークフロー

10.5.1 新しいコードベースの理解

```
> このプロジェクトの概要を教えて
> アーキテクチャを説明して
> 主要なデータモデルは？
```

10.5.2 バグ修正

```
> （エラーログやスタックトレースを貼り付ける）
> このエラーの原因を見つけて
> 修正して
> テストを実行して確認して
```

10.5.3 PR の作成

```
> 変更をコミットして
> PRを作成して
```

10.6 よくある問題と対処法

10.6.1 コンテキストが大きすぎる

`/compact` で圧縮するか `/clear` で新しい会話を開始してください。 1 つの会話で多くのタスクを詰め込みすぎないことが重要です。

10.6.2 権限確認が頻繁

`/permissions` でよく使うツールを許可リストに追加すると、確認ダイアログの頻度を減らせます。 `.claude/settings.json` の `permissions.allow` にパターンを追加します。

10.6.3 Claude が古い情報を参照する

トレーニングデータには期限があります。 最新のドキュメントは直接ファイルを渡すか URL を指定してください：

```
> @docs/api-reference.md を参考にして実装して
```

10.6.4 接続エラー

ネットワーク接続を確認してください。 `claude doctor` で診断を実行できます。

10.7 クイックリファレンス

10.7.1 必須コマンド

```
claude                    # 対話モード開始
claude -p "クエリ"        # ワンショット
claude -c                 # 前回の会話を再開
claude --model opus       # モデル指定
claude --worktree feat-x  # ワークツリーで隔離
```

10.7.2 必須ショートカット

```
Ctrl+C      中断
Ctrl+O      思考過程の表示
Shift+Tab   権限モード切替
Esc × 2     巻き戻し
/           コマンドメニュー
```

10.7.3 3 つの基本パターン

1. `@` でコンテキスト（ファイル参照）を与える
2. Plan モードで安全に探索してから編集する
3. diff を確認してから変更を承認する